

CPE301 – SPRING 2024

Design Assignment 5

Student Name: Joshua Martinez

Student #: 5007531628

Student Email: marti87@unlv.nevada.edu

Primary Github address: <https://github.com/JoshuaMa2003>

Directory: https://github.com/JoshuaMa2003/submission_da

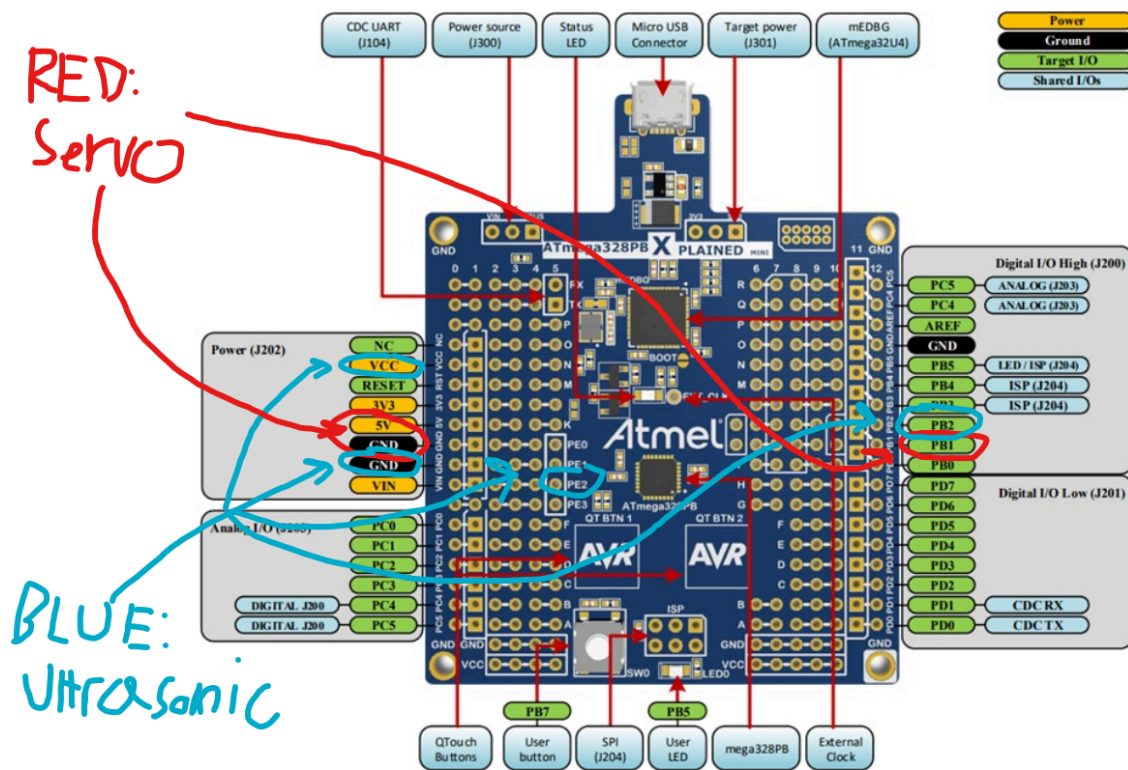
Video Playlist:

https://www.youtube.com/playlist?list=PLZ09MA_uFt63me60r5XWtDUj2m21n4nUP

1. COMPONENTS LIST AND DIAGRAM SHOWING PINS USED

List of Components used

- Microchip Studio Debugger
- Microchip Studio Simulator
- ATmega328PB Microcontroller
- HC-SR04 Ultrasonic Sensor
- SMRAZA S51 Micro Servo
- Processing Radar Software
- Male-to-Male Wires



2. INITIAL CODE OF TASK

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include <stdlib.h>
#include <avr/interrupt.h>
#include <string.h>

#define BAUDRATE 9600 // Define the baud rate for USART
#define BAUD_PRESCALLER (((F_CPU / (BAUDRATE * 16UL))) - 1) // Calculate the baud prescaler

char buffer[5]; // Buffer to hold string representation of integers

// Initialize USART for serial communication
void USART_init(void) {
    UBRR0H = (uint8_t)(BAUD_PRESCALLER >> 8); // Set high byte of the baud rate
    UBRR0L = (uint8_t)(BAUD_PRESCALLER); // Set low byte of the baud rate
    UCSR0B = (1 << RXEN0) | (1 << TXEN0); // Enable receiver and transmitter
    UCSR0C = (3 << UCSZ00); // Set frame format: 8 data bits, no parity, 1 stop bit
}

// Send a byte of data via USART
void USART_send(unsigned char data) {
    while (!(UCSR0A & (1 << UDRE0))); // Wait for empty transmit buffer
    UDR0 = data; // Put data into buffer, sends the data
}

// Send a string via USART
void USART_putstr(char* StringPtr) {
    while (*StringPtr != 0x00) { // Loop through the string until null terminator
        USART_send(*StringPtr); // Send the current character
        StringPtr++; // Increment the pointer
    }
}

#define Trigger_pin PINB2 // Define Trigger pin for ultrasonic sensor

int TimerOverflow = 0; // Overflow counter for timer

// Interrupt service routine for timer overflow
ISR(TIMER3_OVF_vect) {
    TimerOverflow++; // Increment Timer Overflow count
}

// Calculate distance using ultrasonic sensor
double ultrasonic_distance() {
    long count;
    double distance;
    PORTB |= (1 << Trigger_pin); // Send 10us pulse to trigger pin
    _delay_us(10);
    PORTB &= ~(1 << Trigger_pin); // Stop pulse

    TCNT3 = 0; // Clear Timer counter
    TCCR3B = 0x41; // Capture on rising edge, no prescaler
    TIFR3 = 1 << ICF3; // Clear input capture flag
    TIFR3 = 1 << TOV3; // Clear overflow flag

    while ((TIFR3 & (1 << ICF3)) == 0); // Wait for rising edge

    TCNT3 = 0; // Clear Timer counter again for falling edge
    TCCR3B = 0x41; // Capture on falling edge, no prescaler
    TIFR3 = 1 << ICF3; // Clear input capture flag again
    TIFR3 = 1 << TOV3; // Clear overflow flag again
    TimerOverflow = 0; // Clear overflow count

    while ((TIFR3 & (1 << ICF3)) == 0); // Wait for falling edge
    count = ICR3 + (65535 * TimerOverflow); // Read value of capture register
    distance = (double)count / (58 * 16); // Calculate distance in cm
    return distance;
}
```

2. INITIAL CODE OF TASK

```
int main() {
    DDRB = 0x04;           // Set trigger pin as output
    DDRE = 0x00;           // Set PORT E as input (not used here)
    USART_init();          // Initialize USART

    sei();                 // Enable global interrupts
    TIMSK3 = (1 << TOIE3); // Enable Timer3 overflow interrupt
    TCCR3A = 0;            // Normal operation, no PWM

    // Configure TIMER1 for PWM
    TCCR1A |= (1 << COM1A1) | (1 << COM1B1) | (1 << WGM11); // Non-inverted PWM
    TCCR1B |= (1 << WGM13) | (1 << WGM12) | (1 << CS11) | (1 << CS10); // Prescaler=64, Mode 14 (Fast PWM)
    ICR1 = 4999;           // Set top for PWM, fPWM=50Hz (Period = 20ms)

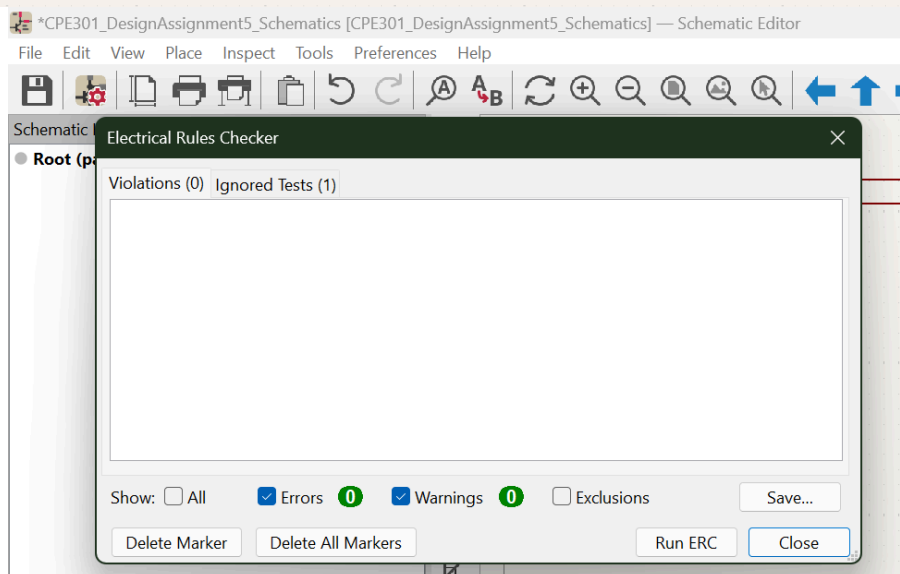
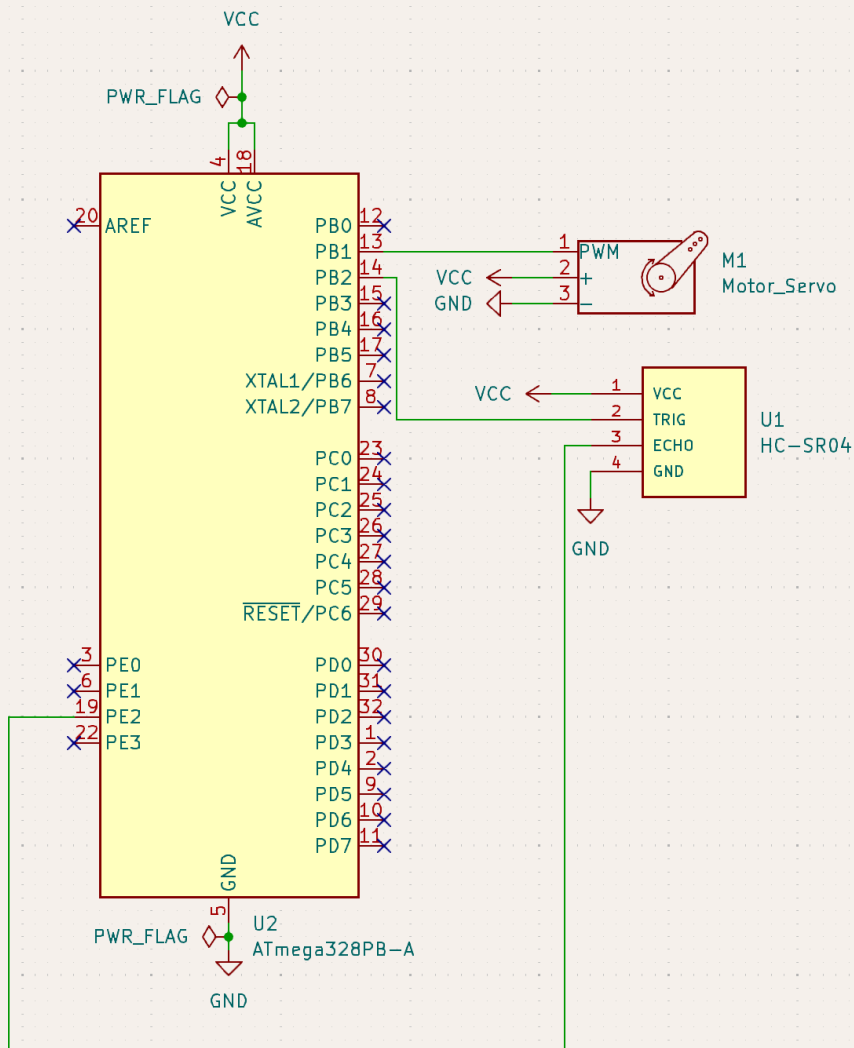
    DDRB |= (1 << PINB1);  // Set PWM pin as output

    while (1) {
        OCR1A = 115;       // Set PWM to represent 0 degree position
        char string[16];
        char angle[16];

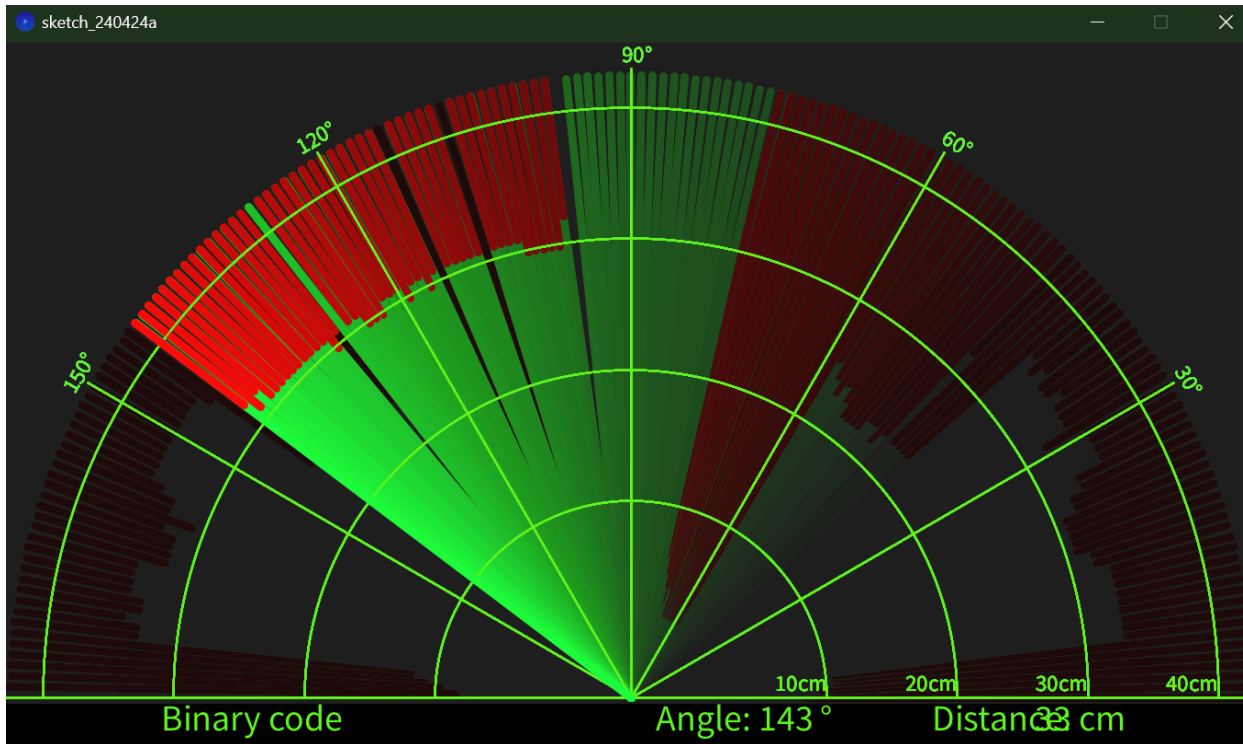
        for (int i = 0; i < 180; i++) { // Increment angle from 0 to 180 degrees
            OCR1A = OCR1A + 5;
            int sweep1 = ultrasonic_distance(); // Measure distance
            itoa(sweep1, string, 10);           // Convert distance to string
            itoa(i, angle, 10);                 // Convert angle to string
            strcat(angle, ",");                 // Append comma
            strcat(string, ".");                // Append dot
            strcat(angle, string);              // Concatenate full string
            USART_putstr(string);               // Send string over USART
            _delay_ms(10);                      // Delay to allow for readings
        }

        for (int i = 0; i < 180; i++) { // Decrement angle from 180 to 0 degrees
            OCR1A = OCR1A - 5;
            int sweep2 = ultrasonic_distance(); // Measure distance
            itoa(sweep2, string, 10);           // Convert distance to string
            itoa(i, angle, 10);                 // Convert angle to string
            strcat(angle, ",");                 // Append comma
            strcat(string, ".");                // Append dot
            strcat(angle, string);              // Concatenate full string
            USART_putstr(string);               // Send string over USART
            _delay_ms(10);                      // Delay to allow for readings
        }
    }
}
```

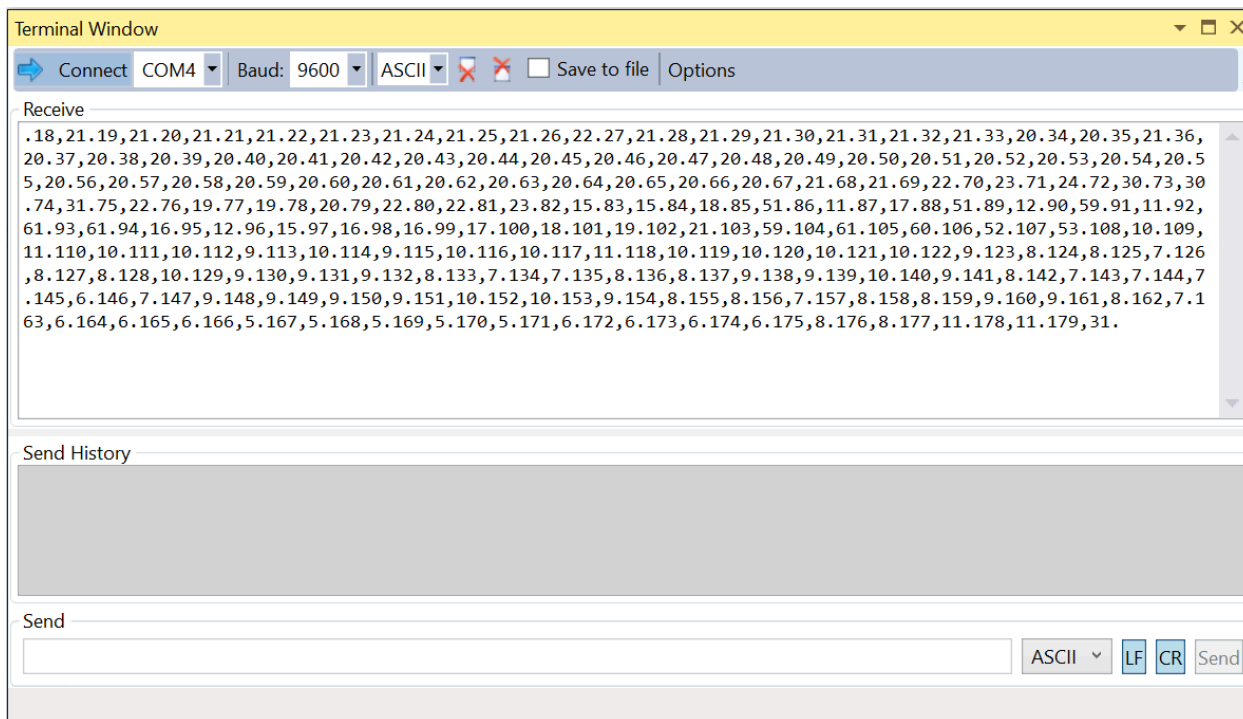

3. KICAD SCHEMATICS FOR TASK



4. SCREENSHOTS OF EACH TASK OUTPUT

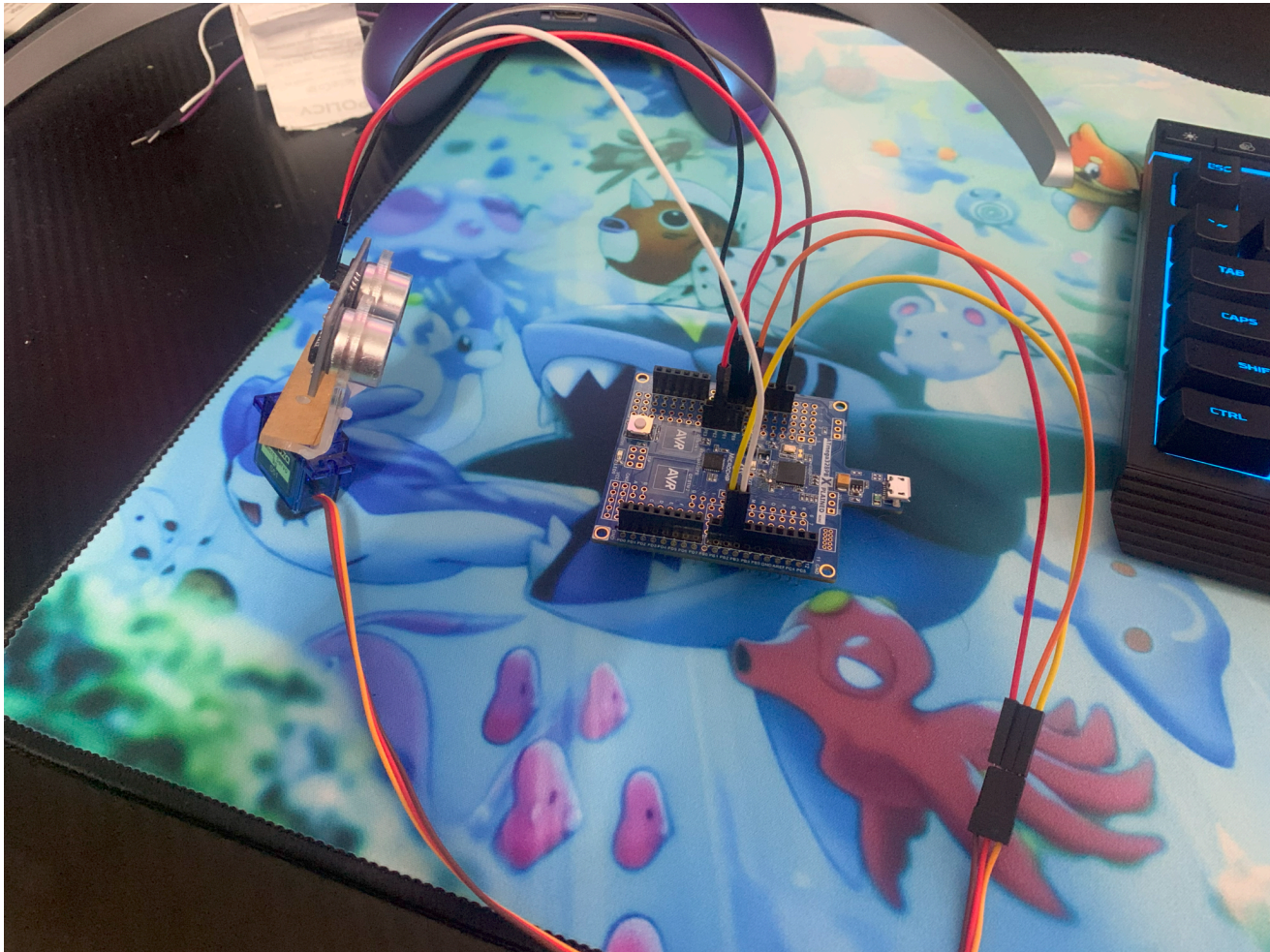


Radar Scan



Terminal Window
Showing Angle and
Distance

5. SCREENSHOTS OF EACH DEMO (BOARD SETUP)



6. VIDEO LINKS OF EACH DEMO

Task Demo: <https://youtu.be/7jeSflf9BL8?feature=shared>

Student Academic Misconduct Policy

<http://studentconduct.unlv.edu/misconduct/policy.html>

"This assignment submission is my own, original work".

JOSHUA MARTINEZ